

C Grundlagen

Datentypen & sizeof

- char (1B), short (2B), int (4B), long (8B), float (4B), double (8B)
- size_t: vorzeichenloser Typ für Grössen (z.B. Rückgabe von sizeof)
- sizeof(Typ) / sizeof(Ausdruck) → Grösse in Bytes, zur **Kompilierzeit** ausgewertet
- uint8_t, int32_t etc. aus <stdint.h> für exakte Breiten

```
int v = 1234;
printf("size=%zd\n", sizeof(v)); // 4
```

Type Casting

```
// Implizit (automatisch):
int i = 'A';           // char → int (65)
double d = 10;         // int → double (10.0)
```

```
// Explizit:
int n = (int)3.9;       // → 3 (truncation, nicht runden!)
double r = (double)5 / 2; // → 2.5 (ohne cast: 2)
void *p = malloc(4);
int *ip = (int *)p;     // void* → int*
```

```
// Fallstricke:
unsigned int u = -1;    // → 4294967295 (wrap-around!)
int a = 200, b = 300;
long c = (long)a * b;   // ohne Cast: int-Overflow!
```

Variablen & Sichtbarkeit

- Lokal:** auf dem Stack, nur innerhalb Block sichtbar
- Global:** im Global/Static-Bereich, gesamtes Programm sichtbar
- Static lokal:** Global/Static-Bereich, nur lokal sichtbar, bleibt erhalten
- extern: Deklaration einer in anderem File definierten Variable
- static bei Funktion/globale Variable → auf aktuelle Übersetzungseinheit begrenzt

Kontrollstrukturen

```
if (cond) { ... } else { ... }
for (int i = 0; i < n; i++) { ... }
while (cond) { ... }
do { ... } while (cond);
switch (x) { case 1: ...; break; default: ...; }
```

Bitoperationen

```
a & b    // AND:  gemeinsame Bits
a | b    // OR:   Bits vereinigen
a ^ b    // XOR:  Bits toggeln
~a       // NOT:  alle Bits invertieren
a << n   // Links-Shift: a * 2^n
a >> n   // Rechts-Shift: a / 2^n
```

```
uint8_t r = 0x00;
r |= (1 << 3);    // Bit 3 setzen   → 0x08
r &= ~(1 << 3);    // Bit 3 löschen → 0x00
r ^= (1 << 3);     // Bit 3 toggeln
if (r & (1 << 3)) { } // Bit 3 prüfen
```

Gefahren mit signed Typen:

- ~, &, |, ^ auf signed int: funktioniert, aber Vorzeichen-Bit wird mitverändert → unerwartet negative Werte
- a >> n auf negativem signed: **implementierungsabhängig** (arithm. oder log. Shift) → unsigned verwenden!
- a << n auf negativem signed oder mit Überlauf: **undefiniertes Verhalten**
- Shift um ≥ Bitbreite (x << 32 bei 32-bit int): **undefiniertes Verhalten**
- Faustregel:** Bitoperationen immer auf unsigned-Typen anwenden: 1u << n statt 1 << n

Systemnahe Programmierung (SNP)

Operator-Vorrangreihenfolge (hoch → tief)

() [] -> .	Klammern, Zugriff
! ~ ++ -- * & (T) sizeof	Unär (rechts-ass.)
* / %	Multiplikativ
+ -	Additiv
<< >>	Bitshift
< <= > >=	Vergleich
== !=	Gleichheit
&/ ^ /	Bitwise AND / XOR / OR
&& /	Logisch AND / OR
?:	Ternär (rechts-ass.)
= += -= ...	Zuweisung (rechts-ass.)

Structs & Enums

```
typedef struct {
    int x;
    char name[32];
} Point;

typedef enum { RED, GREEN, BLUE } Color;
```

```
Point p = { .x = 5, .name = "A" };

Unions
union Data {
    int i;
    float f;
    char bytes[4];
};

union Data d;
d.i = 42; // i, f und bytes teilen denselben Speicher
```

- sizeof(union) = Grösse des **grössten** Members
- Nur ein Member gleichzeitig sinnvoll/gültig
- Typisch: Typkonvertierung auf Byte-Ebene, Protokoll-Parsing

Präprozessor

- #include <stdio.h> – System-Header
- #include "my.h" – eigener Header
- #define MAX 100 – Textersetzung
- #ifdef / #ifndef / #endif – bedingte Kompilierung
- #define SQUARE(x) ((x)*(x)) – Makro (Klammern wichtig!)
- gcc -E file.c → Ausgabe nach Präprozessor

Präprozessor, Compiler, Linker

- Präprozessor:** Textsubstitution (#include, #define, #ifdef)
 - Compiler:** Quellcode → Objektdatei (.o); enthält Maschinencode + offene Symbole
 - Linker:** verbindet Objektdateien + Libraries → ausführbares Programm
- ```
gcc -c hello.c # nur kompilieren → hello.o
gcc -o hello hello.o # linken
gcc -o hello hello.c # alles in einem Schritt
```

Modulare Programmierung

- Declared-Before-Used (DBU):** jeder Name muss vor Verwendung deklariert sein
- One-Definition-Rule (ODR):** jede Funktion/Variable nur einmal definieren
- Header (.h): enthält nur **Deklarationen** (kein Code, keine Variablen-Definition)
- Quelldatei (.c): enthält **Definition** + #include "modul.h"
- static vor Funktion → nur im eigenen File sichtbar (nicht-öffentlich)

Funktionen

Definition & Aufruf

```
// Deklaration (im Header oder oben)
int max(int a, int b);

// Definition
int max(int a, int b) {
 if (a >= b) return a;
 return b;
}
```

Aufruf

```
int v = max(3, 7);
(void)printf("ok\n"); // Rückgabe ignorieren → (void)
```

Parameter by Value

- Parameter werden als **Kopie** übergeben → Original unverändert
- main hat Signatur: int main(void) oder int main(int argc, char \*argv[])
- Rückgabe EXIT\_SUCCESS / EXIT\_FAILURE aus <stdlib.h>

Parameter by Reference

- Adresse übergeben → Funktion kann Original verändern
- ```
void swap(int *a, int *b) {
    int tmp = *a; *a = *b; *b = tmp;
}
```

```
int x = 10, y = 20;
swap(&x, &y); // x==20, y==10
```

const-Qualifikator

Lesehilfe: **von rechts nach links** lesen, * = "Pointer auf".

```
const int x = 5; // x unveränderlich

const int *p; // Pointer auf const int
// → *p nicht änderbar, p selbst schon

int const *p; // identisch (const links/rechts von Typ)

int * const p = &x; // const Pointer auf int
// → p nicht änderbar, *p schon
```

```
const int * const p; // const Pointer auf const int
// → beides unveränderlich
```

- Faustregel:** const rechts vom * → Pointer fix; links → Wert fix
 - Bei Funktionsparametern: const char *s signalisiert, dass die Funktion den String **nicht verändert**
- ```
void print(const char *s); // liest nur → const
void modify(char *s); // schreibt → kein const
```

volatile-Qualifier

- volatile: verhindert Compiler-Optimierung (kein Cachen in Register)
  - Variable kann sich ausserhalb des normalen Kontrollflusses ändern
  - Typisch: Signal-Handler-Flags, Memory-Mapped Hardware-Register
- ```
volatile int flag = 0; // nicht wegoptimieren!
static void handler(int sig) { flag = 1; }

int main(void) {
    signal(SIGINT, handler);
    while (!flag) { /* busy-wait */ }
}
```

```
Variadic Functions
#include <stdarg.h>
int sum(int n, ...) {
    va_list ap;
    va_start(ap, n);
    int s = 0;
    for (int i = 0; i < n; i++) s += va_arg(ap, int);
    va_end(ap);
    return s;
}

Rekursion
int factorial(int n) {
    if (n <= 1) return 1; // Basisfall
    return n * factorial(n - 1); // Rekursionsfall
}

int fib(int n) { // exponentiell ohne Memoization
    if (n <= 1) return n;
    return fib(n-1) + fib(n-2);
}

• Jeder Aufruf: eigener Stack-Frame (lokale Vars + Rücksprungadr.)
• Zu tiefe Rekursion → Stack Overflow
• Tail-Call: Rekursion als letzter Ausdruck → Compiler kann in Schleife umwandeln
```

Funktionspointer

```
int (*fp)(int, int); // fp ist Pointer auf Funktion
fp = max;
int v = fp(3, 7);
```

```
// Als Parameter:
void apply(int *a, int n, int (*f)(int)) {
    for (int i = 0; i < n; i++) a[i] = f(a[i]);
}
```

Arrays & Strings

```
Arrays
int a[5] = {1, 2, 3, 4, 5};
int b[] = {10, 20, 30}; // Grösse aus Initialisierung
int m[3][4]; // 2D-Array: 3 Zeilen, 4 Spalten
• Elemente hintereinander im Speicher
• Array-Name = Adresse des ersten Elements (nicht veränderbar)
• a[i] ≡ *(a + i) (Pointer-Arithmetik)
• Kein Bounds-Checking in C!
```

sizeof bei Arrays

```
int a[5];
sizeof(a) // 20 (Bytes gesamt)
sizeof(a[0]) // 4
sizeof(a)/sizeof(a[0]) // 5 (Anzahl Elemente)
char s[] = "Hello"; // {'H','e','l','l','o','\0'}
sizeof(s) // 6 (inkl. '\0')
strlen(s) // 5 (ohne '\0')
```

Beim Übergeben eines Arrays an eine Funktion “zerfällt” es zum Pointer → sizeof liefert Zeigergrösse (8), nicht Array-Länge!

Programmargumente (argc / argv)

```
int main(int argc, char *argv[]) {
    // argc: Anzahl Argumente (inkl. Programmname)
    // argv[0]: Programmname, argv[1]..argv[argc-1]: Argumente
    // argv[argc] == NULL (Sentinel)
    for (int i = 1; i < argc; i++) {
        printf("arg[%d] = %s\n", i, argv[i]);
    }
    // String → Zahl:
    int n = atoi(argv[1]); // keine Fehlerprüfung
    long l = strtol(argv[1], NULL, 10); // besser:
    Fehlerprüfung via errno
    double d = strtod(argv[1], NULL);
}
```

- argv ist ein Array von char *(C-Strings)
- Aufruf: ./prog foo 42 → argc=3, argv[1]="foo", argv[2]="42"
- Ungültige Argumente: Anzahl prüfen, sonst Segfault bei argv[i]

Umgebungsvariablen

```
#include <stdlib.h>
char *p = getenv("PATH"); // NULL falls nicht gesetzt
if (p) printf("%s\n", p);
```

```
setenv("MY_VAR", "hello", 1); // 1 = überschreiben
unsetenv("MY_VAR");

// Alle Variablen:
extern char **environ;
for (char **e = environ; *e != NULL; e++)
    printf("%s\n", *e); // Format: "NAME=WERT"
• Alternativ: int main(int argc, char *argv[], char *envp[])
```

Strings (char-Arrays)

```
char s[] = "Hello"; // {'H','e','l','l','o','\0'} – veränderbar
char *p = "World"; // String-Literal – read-only! (Schreiben → UB)
• Immer mit \0 terminiert; fehlendes \0 → strlen/printf laufen über
```

Funktion	Header	Verhalten & Edge Cases
strlen(s)	<string.h>	Länge ohne \0; UB wenn kein \0
strcpy(dst, src)	<string.h>	Kopiert inkl. \0; kein Bounds-Check → Buffer Overflow!
strncpy(dst,src,n)	<string.h>	Kopiert max. n Bytes; füllt Rest mit \0; aber: kein \0 wenn src ≥ n Zeichen → manuell setzen!
strcat(dst, src)	<string.h>	Hängt src an dst an; kein Bounds-Check → gefährlich
strncat(dst,src,n)	<string.h>	Hängt max. n Zeichen an; setzt \0; n = verbleibender Platz
strcmp(a, b)	<string.h>	0 = gleich, <0 = a vor b, >0 = a nach b; nie mit == vergleichen!
strncmp(a,b,n)	<string.h>	Wie strcmp, aber max. n Zeichen
strchr(s,c)	<string.h>	Erstes Vorkommen von c; NULL falls nicht gefunden
strstr(s,sub)	<string.h>	Ersten Teilstring finden; NULL falls nicht gefunden
sprintf(buf,fmt,...)	<stdio.h>	Kein Bounds-Check → immer snprintf verwenden!
snprintf(buf,n,fmt,...)	<stdio.h>	Schreibt max. n-1 Zeichen + \0; gibt benötigte Länge zurück

```
// Sicheres Kopieren:
char dst[32];
strncpy(dst, src, sizeof(dst) - 1);
dst[sizeof(dst) - 1] = '\0'; // Sicherheits-\0 immer setzen!
```

```
// Sicheres Anhängen:
strncat(dst, src, sizeof(dst) - strlen(dst) - 1);
```

```
// Strings nie mit == vergleichen (vergleicht Pointer!):
if (strcmp(a, b) == 0) { /* gleich */ }
```

Escape-Sequenzen

Seq.	Bedeutung	Seq.	Bedeutung
\n	Newline (LF)	\t	Tabulator
\r	Carriage Return	\0	Null-Terminator
\\	Backslash	\"	Anführungszeichen
\'	Apostroph	\a	Bell (Signalton)
\b	Backspace	\xHH	Hex (z.B. \x41 = A)

Strings als Funktionsparameter

```
• String-Array zerfällt beim Übergeben zum Pointer → sizeof liefert Zeigergrösse (8), nicht Array-Länge!
• Länge separat übergeben oder strlen nutzen
void print_str(const char *s) { // const: Inhalt read-only
    printf("%s (len=%zu)\n", s, strlen(s));
    // sizeof(s) == 8 (Pointer!), NICHT Arraylänge
}

void fill(char *dst, size_t n, char c) {
    for (size_t i = 0; i + 1 < n; i++) dst[i] = c;
    dst[n - 1] = '\0';
}

char buf[32] = "Hello";
print_str(buf); // Array → const char *
fill(buf, sizeof(buf), 'x'); // Länge explizit übergeben
Pointer
```

Grundlagen

```
int v = 42;
int *p = &v; // p enthält Adresse von v
*p = 100; // Dereferenzierung: v ist jetzt 100
printf("%p\n", (void*)p); // Adresse ausgeben
• void *: typloser Pointer, kompatibel mit allen Pointer-Typen
• NULL: Null-Pointer (0), kein gültiges Ziel
Pointer-Arithmetik
int a[] = {10, 20, 30, 40};
int *p = a;
p++; // p zeigt jetzt auf a[1]
p += 2; // p zeigt auf a[3]
*(p - 1) // a[2]
p - a // Differenz = 2 (Anzahl Elemente)
• Addition/Subtraktion skaliert automatisch mit sizeof(Typ)
Struct-Pointer
typedef struct { int x, y; } Point;
Point pt = {3, 5};
Point *pp = &pt;
pp->x = 10; // identisch mit (*pp).x = 10
Mehrdimensionale Arrays & Pointer
int m[3][4]; // Array von 3 Pointern auf int[4]
// m[i][j] == (*(m + i) + j)
// Array von Pointern (verschieden lang):
char *names[] = {"Alice", "Bob", "Charlie"};
```

I/O & Standard Library

```
stdio.h
printf("fmt %d %s\n", 42, "hi"); // stdout
fprintf(stderr, "Fehler!\n"); // stderr
scanf("%d", &n); // stdin
fgets(buf, sizeof(buf), stdin); // Zeile lesen (sicher)
snprintf(buf, sz, "fmt", ...); // sicheres sprintf
printf / scanf Formatzeichen
```

Spez.	Typ	Bedeutung
%d / %i	int	Ganzzahl dezimal
%u	unsigned int	Ganzzahl dezimal vorzeichenlos
%ld	long	Long dezimal
%lld	long long	Long long dezimal
%zu	size_t	Grösse (sizeof, strlen)
%f	double	Fliesskomma dezimal
%lf	double	Fliesskomma (scanf)
%e	double	Exponentialdarstellung
%g	double	Kürzer von %f / %e
%c	char	Einzelnes Zeichen
%s	char *	C-String (bis \0)
%p	void *	Pointer-Adresse (hex)
%x / %X	unsigned int	Hexadezimal klein/gross
%o	unsigned int	Oktal
%%	-	Literal %

- Breite & Präzision: %5d (Breite 5), %.2f (2 Nachkommastellen), %8.3f
- Links-ausrichten: %-10s; Nullen auffüllen: %05d
- scanf: benötigt Adresse → scanf("%d", &n), String: scanf("%s", buf)

```
stdlib.h (Auswahl)
#include <stdlib.h>
abs(-5) // → 5 (int; labs() für long)
rand() // Zufallszahl [0, RAND_MAX]
srand(time(NULL)); // Seed setzen (einmalig!)
```

```
// Sortieren & Suchen:
int cmp(const void *a, const void *b) {
    return *(int*)a - *(int*)b;
}

int arr[] = {3, 1, 2};
qsort(arr, 3, sizeof(int), cmp); // in-place sortieren
int key = 2;
int *f = bsearch(&key, arr, 3, sizeof(int), cmp);
// bsearch: NULL falls nicht gefunden; Array muss sortiert sein!
File I/O (stdio)
FILE *f = fopen("file.txt", "r"); // "r", "w", "a", "rb"...
if (!f) { perror("fopen"); exit(1); }
char line[256];
while (fgets(line, sizeof(line), f)) { ... }
fclose(f);
fprintf(f, "Hello %d\n", 42);
fseek(f, 0, SEEK_SET); // Position setzen
```

```
System Call I/O
#include <fcntl.h>
#include <unistd.h>
int fd = open("file", O_RDONLY); // O_WRONLY, O_RDWR, O_CREAT
if (fd == -1) { perror("open"); exit(EXIT_FAILURE); }
```

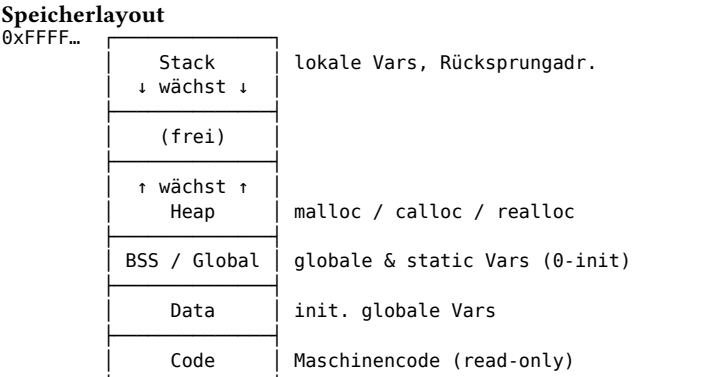
```
ssize_t n = read(fd, buf, sizeof(buf));
if (n == -1) { perror("read"); close(fd); exit(EXIT_FAILURE); }
// n == 0: EOF
```

```
if (write(fd, buf, n) == -1) { perror("write"); }
```

```
if (close(fd) == -1) { perror("close"); }
```

```
if (lseek(fd, 0, SEEK_SET) == -1) { perror("lseek"); }
• stdin=0, stdout=1, stderr=2 (File Deskriptoren)
```

Dynamische Allokierung



```
0x0000...
malloc / calloc / realloc / free
#include <stdlib.h>
// Alloziert size Bytes (uninitialisiert)
void *malloc(size_t size);
```

```
// Alloziert nitems*size Bytes, auf 0 gesetzt
void *calloc(size_t nitems, size_t size);
```

```
// Grösse ändern (kann Inhalt umkopieren)
void *realloc(void *ptr, size_t size);
```

```
// Speicher freigeben (ptr danach nicht verwenden!)
void free(void *ptr);
int *p = malloc(10 * sizeof(int));
if (!p) { perror("malloc"); exit(EXIT_FAILURE); }
p[0] = 42;
free(p);
p = NULL; // sicherer: Verhindert versehentlichen Zugriff auf freigegebenen Speicher
```

Häufige Fehler

- **Memory Leak:** free() vergessen
- **Doppelt free:** Heap-Korruption
- **Stack Overflow:** zu tiefe Rekursion / grosse lokale Arrays
- **Buffer Overflow:** Schreiben ausserhalb allozierter Grenzen
- Immer Rückgabewert von malloc() prüfen (== NULL)

Computersysteme & OS

Hardware-Schichtenmodell

- Anwendung
- System Library (glibc)
 - System Calls
- Betriebssystem-Kernel
- Hardware (CPU, RAM, I/O)

Kernel- und User-Modus

- **Kernel-Modus:** alle Operationen erlaubt (Ring 0)
 - **User-Modus:** eingeschränkt, kein direkter Hardware-Zugriff
 - Übergang via **System Call** (syscall)
 - syscall(number, ...) → bei Fehler return -1, errno gesetzt
 - Ca. 300 System Calls unter Linux (man 2 syscalls)
- // Fehlerhandling-Makro

```
#define PERROR_AND_EXIT(M) \
do { perror(M); exit(EXIT_FAILURE); } while(0)
```

Memory Management Unit (MMU) & MPU

- **MPU** (Memory Protection Unit): überwacht Adress-Bus, löst Exception bei unautorisiertem Zugriff aus
- **MMU** (Memory Management Unit): übersetzt virtuelle → physikalische Adressen; beinhaltet MPU-Funktionalität
- Virtuelles Memory: jeder Prozess hat privaten (virtuellen) Adressraum
- Physischer Speicher kann auf Massenspeicher ausgelagert werden (Swap)

Standards

- **POSIX:** definiert C-API zu Unix-ähnlichen Betriebssystemen
- **FHS** (Filesystem Hierarchy Standard): wo liegen welche Files (/bin, /etc, /usr, /var, ...)
- **glibc:** C Standard Library + C POSIX Library + GNU Extensions
- Compiler-Standard wählen: gcc -std=c11 oder -std=gnu11 (Default)

Filesystem & I/O

“Everything is a File”

- Reguläre Files, Directories, Devices, Pipes, Sockets → alles Files
- Zugriff: öffnen → lesen/schreiben → schliessen
- **File Desriptor** (fd): Integer-ID für geöffnetes File

Inode & Links

- **Inode:** Verwaltungseinheit eines Files (Metadaten: Grösse, Besitzer, Timestamps, Ort auf Disk) – **nicht** der Dateiname
- **Hard-Link:** Verzeichniseintrag → Inode; mehrere Links auf selbe Inode möglich; erst wenn Zähler = 0 wird File gelöscht
- **Symlink:** spezielles File mit Pfad als Inhalt; kann auf andere Filesysteme zeigen

```
ln file hardlink # Hard-Link erstellen
ln -s file symlink # Symlink erstellen
```

Dateisystem-Hierarchie (Auswahl)

Pfad	Inhalt
/bin	Systemkommandos
/etc	Konfigurationsdateien
/home	Benutzerverzeichnisse
/dev	Device-Files
/proc, /sys	virtuelle Filesysteme
/tmp	temporäre Dateien

Spezielle Files

- **Character Device:** Byte-weiser Zugriff (z.B. /dev/tty, Tastatur)
- **Block Device:** Block-weiser Zugriff (z.B. /dev/sda, Festplatte)
- **Named Pipe:** IPC zwischen Prozessen
- **Socket:** Netzwerk-/lokale IPC

Directory-Operationen & stat

```
#include <dirent.h>
#include <sys/stat.h>

DIR *d = opendir("/tmp");
struct dirent *e;
while ((e = readdir(d)) != NULL)
    printf("%s\n", e->d_name); // inkl. "." und ".."
closedir(d);
```

```
struct stat st;
stat("file.txt", &st); // lstat() folgt Symlink NICHT
printf("Grösse: %ld\n", (long)st.st_size);
printf("Rechte: %o\n", st.st_mode & 0777);
// Typprüfung:
S_ISREG(st.st_mode) // reguläres File
S_ISDIR(st.st_mode) // Verzeichnis
S_ISLNK(st.st_mode) // Symlink (nur mit lstat)
```

Stream Buffering

- **Vollgepuffert:** Ausgabe bei vollem Puffer oder fflush()
- **Zeilengepuffert:** Ausgabe bei \n (stdout im Terminalmode)
- **Ungepuffert:** sofort (stderr)
- fflush(stdout) – Puffer leeren

Prozesse & Threads

Multi-Tasking

- **Batch:** Tasks hintereinander
- **Kooperativ:** Task gibt Kontrolle selbst ab
- **Präemptiv:** Scheduler unterbricht zwangsweise (z.B. via HW-Timer)
- **Scheduler:** entscheidet welche Task als nächstes läuft (round-robin, priority-driven, ...)
- **Kontext-Switch:** CPU-Register + MMU-Zustand sichern/wiederherstellen

Prozess

- Programm in Ausführung mit: Code, Daten, virtuellem Memory, Ressourcen (Files etc.)
- **Eigenes** virtuelles Memory (isoliert von anderen Prozessen)
- Zustände: running → ready → blocked → terminated
- Prozess-Kontrollblock (PCB): OS-interne Verwaltungsstruktur

Thread

- Leichtgewichtiger Kontrollfluss **innerhalb** eines Prozesses
- Teilt sich Memory + Ressourcen mit anderen Threads desselben Prozesses
- Günstiger Kontext-Switch (kein MMU-Umkonfigurieren)
- Kein Speicherschutz zwischen Threads → Synchronisation nötig
- Eigener Stack + Register + Thread-ID

exec & system

```
// Neues Programm im Kindprozess laden:
char *argv[] = {"ls", "-l", NULL};
execv("/bin/ls", argv); // returnt nur bei Fehler
```

```
// Komfortfunktion (fork+exec+wait intern):
int ret = system("/bin/ls -l");
int code = WEXITSTATUS(ret);
```

```
// stdout des Unterprozesses lesen:
FILE *f = popen("df -k", "r");
// ... fgets(line, sizeof(line), f) ...
pclose(f);
```

Prozess-API

```
#include <unistd.h>
#include <sys/wait.h>

pid_t fork();
// fork(): -1=Fehler, 0=Kindprozess, >0=PID des Kinds

// im Elternprozess:
int wstatus;
pid_t wpid = waitpid(cpid, &wstatus, 0); // blockierend
int exitcode = WEXITSTATUS(wstatus);
```

```
exit(EXIT_SUCCESS); // Prozess terminieren
getpid(); // eigene PID
getppid(); // Eltern-PID
// Vollständiges Beispiel
pid_t cpid = fork();
if (cpid == -1) perror_and_exit("fork");
if (cpid > 0) { // Elternprozess
    int ws;
    waitpid(cpid, &ws, 0);
    printf("Kind exitcode: %d\n", WEXITSTATUS(ws));
    exit(EXIT_SUCCESS);
} else { // Kindprozess
    sleep(1);
    exit(42);
}
```

Thread-API (pthreads)

```
#include <pthread.h>
// Kompilieren mit: gcc ... -lpthread

// Threadfunktion-Signatur:
void *worker(void *arg) {
    int *val = (int*)arg;
    // ... Arbeit ...
    static int ret = 42;
    return &ret; // Rückgabewert
}
```

```
pthread_t tid;
// Thread starten:
pthread_create(&tid, NULL, worker, &arg);
// Auf Ende warten + Rückgabewert holen:
void *retval;
pthread_join(tid, &retval);
// ODER: Ressourcen automatisch freigeben:
pthread_detach(tid);
• Fehler-Check: int r = pthread_create(...); if (r) { errno=r; perror(...); }
```

Zombie & Orphan

- **Zombie:** Kind terminiert, Eltern hat noch kein wait() gemacht → bleibt als Struktur im OS
- **Orphan:** Elternprozess terminiert, bevor Kind → wird von init (PID 1) adoptiert

Thread-Synchronisation

Race Condition & Critical Section

- **Race Condition:** Ergebnis hängt von Ausführungsreihenfolge ab → Fehler
- **Critical Section:** Codebereich mit exklusivem Zugriff auf gemeinsame Ressource
- **Mutual Exclusion (Mutex):** nur eine Task gleichzeitig in der Critical Section

Mutex

```
#include <pthread.h>
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
// oder:
pthread_mutex_init(&mutex, NULL);
```

```
pthread_mutex_lock(&mutex); // blockiert bis Mutex frei
pthread_mutex_unlock(&mutex); // Mutex freigeben
// Nicht-blockierend EBusy falls schon gesperrt:
int r = pthread_mutex_trylock(&mutex);
```

```
pthread_mutex_destroy(&mutex);
• Lock/Unlock immer im selben Thread
• Kein unnötiges Lock-Halten (kostet Zeit)
• Rekursive Mutex: pthread_mutexattr_settype(&attr, PTHREAD_MUTEX_RECURSIVE)
```

Semaphore

```
#include <semaphore.h>
sem_t sem;
sem_init(&sem, 0, 0); // 0=thread-shared, Startwert=0
```

```
sem_wait(&sem); // Down/P: Zähler=0 → blockieren
sem_post(&sem); // Up/V: Zähler erhöhen, wartende Task freigeben
sem_destroy(&sem);
```

- Typische Anwendung: Task wartet auf Ergebnis einer anderen Task (Signalisierung)
- Startwert 0 → wartende Task blockiert bis sem_post von anderer Task

Deadlock

- Entsteht wenn Task A auf Task B wartet, B gleichzeitig auf A
- Verhindert durch: konsistente Lock-Reihenfolge, Timeout, Lock-Hierarchie
- Symptom: Programm hängt für immer

Coffman-Bedingungen (alle 4 müssen gleichzeitig gelten)

- **Mutual Exclusion:** Ressource kann nur von einer Task genutzt werden
- **Hold & Wait:** Task hält Ressource und wartet auf weitere
- **No Preemption:** Ressource kann Task nicht entzogen werden
- **Circular Wait:** zyklische Kette von Tasks, jede wartet auf Ressource der nächsten

Starvation

- Thread wartet unbegrenzt lange, weil andere Threads bevorzugt werden
- Ursachen: unfaire Scheduler-Politik, Prioritätsinversion, immer neue höher-priorisierte Threads
- Verhindert durch: faire Locks (PTHREAD_MUTEX_ERRORCHECK), Priority Inheritance, Aging (Priorität steigt mit Wartezeit)

Weitere Synchronisationsmittel

```
• Monitor / Condition Variable: Mutex + Bedingung
pthread_cond_t cond = PTHREAD_COND_INITIALIZER;
pthread_cond_wait(&cond, &mutex); // Mutex freigeben + warten
pthread_cond_signal(&cond); // eine wartende Task aufwecken
pthread_cond_broadcast(&cond); // alle aufwecken
• Barrier: warten bis N Tasks angekommen sind
pthread_barrier_t b;
pthread_barrier_init(&b, NULL, N);
pthread_barrier_wait(&b); // blockiert bis N Tasks hier sind
```


Inter-Process Communication (IPC)

Übersicht

Mechanismus	Art	Synchronisation
Signal	Ereignis	implizit
Pipe	Datenstrom	implizit
Message Queue	Nachrichten	implizit
Socket	Datenstrom	implizit
Shared Memory	Speicher	explizit
Shared File	Datei	explizit

POSIX Signals

```
#include <signal.h>
// Signal senden:
kill(pid, SIGTERM); // SIGINT, SIGKILL, SIGTERM, ...
```

```
// Signal-Handler registrieren:
static void handler(int sig) { /* async-signal-safe */ }
signal(SIGINT, handler);
// oder (robuster):
struct sigaction sa = { .sa_handler = handler };
sigaction(SIGINT, &sa, NULL);
```

Signal	Default	Bedeutung
SIGINT	Term	Ctrl+C
SIGTERM	Term	Terminierung
SIGKILL	Term	Kill (nicht abfangbar)
SIGQUIT	Core	Ctrl+\
SIGSEGV	Core	Ungültiger Speicherzugriff
SIGALRM	Term	Timer
SIGCHLD	Ign	Kindprozess terminiert
SIGSTOP	Stop	Prozess stoppen (nicht abfangbar)
SIGCONT	Cont	Prozess fortsetzen

POSIX Pipes

```
#include <unistd.h>
int fd[2];
pipe(fd); // fd[0]=lesen, fd[1]=schreiben
// Verwendung typisch: nach fork()
// Eltern schreibt fd[1], Kind liest fd[0] (oder umgekehrt)
write(fd[1], "hello", 5);
read(fd[0], buf, sizeof(buf));
close(fd[0]); close(fd[1]);
• Unidirektional, FIFO, blockierend
• Named Pipe: mkfifo("/tmp/pipe", 0600) → per Pfad zugreifbar
```

POSIX Message Queue

```
#include <mqueue.h>
// Erstellen: mq_open(name, O_CREAT|O_RDWR, 0600, &attr)
// Senden: mq_send(mq, buf, len, prio)
// Empfangen: mq_receive(mq, buf, len, &prio)
// Schliessen/Löschen: mq_close, mq_unlink
// Kompilieren: -lrt
```

POSIX Shared Memory

```
#include <sys/mman.h> // mmap, munmap
// shm_open/shm_unlink benötigen: -lrt
int fd = shm_open("/myshm", O_CREAT|O_RDWR, 0600);
ftruncate(fd, sizeof(int)); // Grösse setzen
int *ptr = mmap(NULL, sizeof(int),
PROT_READ|PROT_WRITE, MAP_SHARED, fd, 0);
close(fd);
*ptr = 42; // Verwenden
munmap(ptr, sizeof(int)); // Freigeben
shm_unlink("/myshm"); // Segment löschen
• Synchronisation nötig! (Mutex oder Semaphore)
• MAP_ANONYMOUS | MAP_SHARED + fork() → ohne Namen
```

POSIX Sockets

```
#include <sys/socket.h>
#include <netinet/in.h>
// Server:
int s = socket(AF_INET, SOCK_STREAM, 0); // TCP
bind(s, (struct sockaddr*)&addr, sizeof(addr));
listen(s, 5);
int c = accept(s, NULL, NULL);
read(c, buf, sz); write(c, buf, sz); close(c);

// Client:
int s = socket(AF_INET, SOCK_STREAM, 0);
connect(s, (struct sockaddr*)&addr, sizeof(addr));
write(s, buf, sz); read(s, buf, sz); close(s);
• AF_UNIX für lokale Sockets (kein Netzwerk)
• SOCK_DGRAM für UDP (verbindungslos)
```

Make & Build

Makefile-Grundstruktur

```
CC = gcc
CFLAGS = -Wall -Wextra -std=gnu11 -g
```

```
all: myprog
```

```
myprog: main.o modul.o
$(CC) $(CFLAGS) -o @$^
```

```
%.o: %.c
$(CC) $(CFLAGS) -c $<
```

```
clean:
rm -f *.o myprog
• $@: Zielfeile
• $^: alle Abhängigkeiten
• $<: erste Abhängigkeit
• Einrückung mit Tab (nicht Spaces)!
• make -j4 – parallel bauen
```

Code Quality & Test

Defensive Programmierung

```
// assert für Invarianten (deaktivierbar mit -DDEBUG)
#include <assert.h>
assert(ptr != NULL);
assert(n > 0);
```

```
// Rückgabewerte IMMER prüfen
FILE *f = fopen("x", "r");
if (!f) { perror("fopen"); exit(EXIT_FAILURE); }

Nützliche gcc-Flags
-Wall -Wextra # Warnungen einschalten
-Werror # Warnungen als Fehler
-g # Debug-Symbole
-O0 # keine Optimierung (Default, gut zum Debuggen)
-O2 # empfohlene Optimierung für Release
-O3 # aggressive Optimierung
-Os # Grösse minimieren
-fsanitize=address # AddressSanitizer (Speicherfehler)
-fsanitize=thread # ThreadSanitizer (Race Conditions)
-std=gnu11 # C11 Standard
```

Valgrind

```
valgrind --leak-check=full ./myprog
valgrind --tool=helgrind ./myprog # Thread-Fehler
```

GDB (Debugger)

```
gcc -g -o prog prog.c # Debug-Symbole einbetten
gdb ./prog
(gdb) break main # Breakpoint bei Funktion
(gdb) break datei.c:42 # Breakpoint bei Zeile
(gdb) run [args] # Programm starten
(gdb) next (n) # nächste Zeile, kein Step-in
(gdb) step (s) # in Funktion einsteigen
(gdb) continue (c) # bis nächsten Breakpoint
(gdb) print var # Variable ausgeben
(gdb) backtrace (bt) # Aufruf-Stack anzeigen
(gdb) info locals # lokale Variablen
(gdb) quit (q) # GDB beenden
```

errno & perror

```
#include <errno.h>
// errno: globale Variable, nach Fehler gesetzt
// perror("msg"): gibt "msg: Fehlermeldung\n" auf stderr
// strerror(errno): Fehlermeldung als String
if (open(...) == -1) {
fprintf(stderr, "open: %s\n", strerror(errno));
}
```

sizeof-Referenz

Typ	Bytes	Min	Max
char	1	-128	127
unsigned char	1	0	255
short	2	-32 768	32 767
unsigned short	2	0	65 535
int	4	-2 147 483 648	2 147 483 647
unsigned int	4	0	4 294 967 295
long	8	-9.2 × 10 ¹⁸	9.2 × 10 ¹⁸
unsigned long	8	0	1.8 × 10 ¹⁹
float	4	1.2 × 10 ⁻³⁸	3.4 × 10 ³⁸
double	8	2.2 × 10 ⁻³⁰⁸	1.8 × 10 ³⁰⁸
pointer	8	–	–
size_t	8	0	18.4 × 10 ¹⁸
int8_t	1	-128	127
uint8_t	1	0	255
int16_t	2	-32 768	32 767
uint16_t	2	0	65 535
int32_t	4	-2 147 483 648	2 147 483 647
uint32_t	4	0	4 294 967 295
int64_t	8	-9.2 × 10 ¹⁸	9.2 × 10 ¹⁸
uint64_t	8	0	1.8 × 10 ¹⁹

- Größen gelten für 64-bit Linux (x86-64); long ist auf Windows 32-bit 4 Bytes
- Konstanten aus <limits.h>: INT_MAX, UINT_MAX, LONG_MIN, ...
- Konstanten aus <float.h>: FLT_MAX, DBL_MAX, FLT_EPSILON, ...

Array Decay: Arrays zerfallen beim Übergeben an eine Funktion zu einem

Pointer → sizeof liefert dann 8 (Zeigergrösse), nicht die Array-Länge!

```
void f(int a[]) {
    sizeof(a); // 8 (Pointer!) – NICHT sizeof des Arrays
}

int arr[10];
sizeof(arr); // 40 (korrekt, da im selben Scope)
f(arr); // arr zerfällt zu int* → sizeof in f() = 8
→ Arraylänge immer als separaten Parameter übergeben: void f(int *a,
size_t n)
```

Prüfungs-Snippets

String umkehren

```
void reverse(char *s) {
    int l = 0, r = strlen(s) - 1;
    while (l < r) {
        char tmp = s[l]; s[l] = s[r]; s[r] = tmp;
        l++; r--;
    }
}
```

Ziffer extrahieren (Modulo)

```
int n = 12345;
int letzte = n % 10; // 5
int zweitletzte = (n / 10) % 10; // 4
int quersumme(int n) {
    if (n < 0) n = -n; // negative Zahlen
    int s = 0;
    while (n != 0) { s += n % 10; n /= 10; }
    return s;
}
```

Palindrom prüfen

```
int is_palindrom(const char *s) {
    int l = 0, r = strlen(s) - 1;
    while (l < r)
        if (s[l++] != s[r--]) return 0;
    return 1;
}
```

Zeichen zählen / suchen

#include <ctype.h>

```
int count_char(const char *s, char c) {
    int n = 0;
    while (*s) if (*s++ == c) n++;
    return n;
}
```

// Grossbuchstaben → Kleinbuchstaben:
for (int i = 0; s[i]; i++) s[i] = tolower(s[i]);
// weitere: toupper(), isdigit(), isalpha(), isspace()

Array: Min / Max / Summe

```
int min = a[0], max = a[0], sum = 0;
for (int i = 0; i < n; i++) {
    if (a[i] < min) min = a[i];
    if (a[i] > max) max = a[i];
    sum += a[i];
}
```

double avg = (double)sum / n;

Bubble Sort

```
for (int i = 0; i < n - 1; i++)
    for (int j = 0; j < n - i - 1; j++)
        if (a[j] > a[j+1]) {
            int t = a[j]; a[j] = a[j+1]; a[j+1] = t;
        }
```

Primzahl prüfen

```
int is_prime(int n) {
    if (n < 2) return 0;
    for (int i = 2; i * i <= n; i++)
        if (n % i == 0) return 0;
    return 1;
}
```

GGT (Euklid) & KGV

```
int ggt(int a, int b) {
    while (b) { int t = b; b = a % b; a = t; }
    return a;
}
```

int kgv(int a, int b) { return a / ggt(a, b) * b; }

Binäre Suche (sortiertes Array)

```
int binsearch(int *a, int n, int key) {
    int lo = 0, hi = n - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2; // kein Overflow
        if (a[mid] == key) return mid;
        else if (a[mid] < key) lo = mid + 1;
        else hi = mid - 1;
    }
    return -1; // nicht gefunden
}
```

Dynamische Liste (Linked List)

typedef struct Node { int val; struct Node *next; } Node;

```
Node *push(Node *head, int val) {
    Node *n = malloc(sizeof(Node));
    n->val = val; n->next = head;
    return n; // neuer Kopf
}

void print_list(Node *head) {
    for (; head; head = head->next)
        printf("%d ", head->val);
}

void free_list(Node *head) {
    while (head) { Node *t = head->next; free(head); head = t; }
}
```

ASCII Tabelle

D	H	Z	D	H	Z	D	H	Z	D	H	Z
0	00	NUL	32	20	SP	64	40	@	96	60	`
1	01	SOH	33	21	!	65	41	A	97	61	a
2	02	STX	34	22	"	66	42	B	98	62	b
3	03	ETX	35	23	#	67	43	C	99	63	c
4	04	EOT	36	24	\$	68	44	D	100	64	d
5	05	ENQ	37	25	%	69	45	E	101	65	e
6	06	ACK	38	26	&	70	46	F	102	66	f
7	07	BEL	39	27	'	71	47	G	103	67	g
8	08	BS	40	28	(	72	48	H	104	68	h
9	09	HT	41	29	)	73	49	I	105	69	i
10	0A	LF	42	2A	*	74	4A	J	106	6A	j
11	0B	VT	43	2B	+	75	4B	K	107	6B	k
12	0C	FF	44	2C	,	76	4C	L	108	6C	l
13	0D	CR	45	2D	-	77	4D	M	109	6D	m
14	0E	SO	46	2E	.	78	4E	N	110	6E	n
15	0F	SI	47	2F	/	79	4F	O	111	6F	o
16	10	DLE	48	30	0	80	50	P	112	70	p
17	11	DC1	49	31	1	81	51	Q	113	71	q
18	12	DC2	50	32	2	82	52	R	114	72	r
19	13	DC3	51	33	3	83	53	S	115	73	s
20	14	DC4	52	34	4	84	54	T	116	74	t
21	15	NAK	53	35	5	85	55	U	117	75	u
22	16	SYN	54	36	6	86	56	V	118	76	v
23	17	ETB	55	37	7	87	57	W	119	77	w
24	18	CAN	56	38	8	88	58	X	120	78	x
25	19	EM	57	39	9	89	59	Y	121	79	y
26	1A	SUB	58	3A	:	90	5A	Z	122	7A	z
27	1B	ESC	59	3B	;	91	5B	[	123	7B	{
28	1C	FS	60	3C	<	92	5C	\	124	7C	
29	1D	GS	61	3D	=	93	5D	]	125	7D	}
30	1E	RS	62	3E	>	94	5E	^	126	7E	~
31	1F	US	63	3F	?	95	5F	_	127	7F	DEL